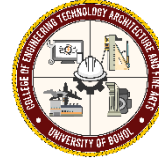




UNIVERSITY OF BOHOL



Tagbilaran City, Bohol, Philippines

COLLEGE OF ENGINEERING, TECHNOLOGY, ARCHITECTURE AND FINE ARTS

COMPUTER ENGINEERING

MICROPROCESSOR LEC/LAB

CPEP 324A

Ball and Beam Balancing System using LQR Control with Kalman Filtering - Documentation

Submitted by:

SUPREMO, EDEAN R.

QUIJADA, LEONARD ANTHONY

Abstract

This documentation presents a complete technical analysis and implementation of a ball-and-beam balancing control system deployed on an Arduino Nano microcontroller (ATmega328P, 16 MHz). The system employs a Linear Quadratic Regulator (LQR) control strategy combined with a scalar Kalman filter for robust state estimation and measurement noise reduction. An HC-SR04 ultrasonic sensor provides continuous distance measurements, which are processed through the Kalman filter to estimate ball position. Velocity is derived via numerical differentiation with exponential smoothing. A servo motor (MG996R) tilts the beam to guide the ball toward a target equilibrium position. The controller is implemented in pure AVR C without Arduino libraries, utilizing Timer1 for 50 Hz servo PWM generation and Timer2 for precise millisecond timing. Experimental results demonstrate stable convergence to the target position with significantly reduced sensor noise, validating the LQR-Kalman approach on a resource-constrained embedded platform.

Introduction

The ball-and-beam system is a classic benchmark problem in control theory education and research. It serves as an ideal testbed for demonstrating fundamental principles of nonlinear system dynamics, state estimation under measurement noise, feedback control design, and real-time embedded control implementation. These principles are directly applicable in aerospace, robotics, and industrial automation systems.

Traditional approaches to this problem employ PID (Proportional-Integral-Derivative) control, which, while intuitive and easy to implement, requires labor-intensive manual tuning and relies on only the position error as feedback. This project instead

implements a Linear Quadratic Regulator (LQR) — an optimal control strategy that minimizes a quadratic cost function — combined with a Kalman filter for full-state estimation.

The project demonstrates that modern optimal control techniques are not confined to high-end computational platforms but are achievable on resource-constrained microcontrollers such as the Arduino Nano, bridging the gap between theory and practical embedded implementation.

Objectives

1. Design and implement an LQR controller that stabilizes the ball at a target equilibrium position on the beam.
2. Apply Kalman filtering to reduce measurement noise and estimate unmeasured states (velocity).
3. Implement the complete control system on the Arduino Nano platform (ATmega328P @ 16 MHz).
4. Validate system performance through simulation and experimental testing.
5. Document all aspects of the project including hardware, algorithms, mathematical derivations, and source code.

Scope and Limitations

This project covers the design, implementation, and testing of the ball-and-beam balancing system within the following scope:

- Ball position range: 3.9 cm to 12.9 cm along the beam.

- Control loop rate: 50 Hz (20 ms sampling period).
- Platform: Arduino Nano, ATmega328P, 16 MHz, 1 KB SRAM.
- The system assumes small-angle approximation for linearization ($\sin \theta \approx \theta$).
- Rolling friction, air drag, and servo nonlinearity at small angles are treated as minor disturbances.

Project Overview

The Ball and Beam Balancing System is a closed-loop control system in which a ball is made to rest at a specified position along a horizontal beam. The beam is actuated by a servo motor, and an ultrasonic sensor continuously measures the ball's distance from a fixed reference. The microcontroller runs the control algorithm, computes the required servo angle, and drives the servo in real time.

System Architecture

The overall system is organized into four functional layers:

- Sensing Layer: HC-SR04 ultrasonic distance sensor measuring ball position.
- Estimation Layer: Scalar Kalman filter for position estimate; numerical differentiation + exponential smoothing for velocity.
- Control Layer: LQR controller computing optimal servo command from estimated states.
- Actuation Layer: MG996R servo motor receiving 50 Hz PWM signal to tilt the beam.

Components Used

All components were selected for their availability, compatibility with 5V logic, and suitability for real-time embedded control applications.

Component	Model / Specification	Role in System
Microcontroller	Arduino Nano (ATmega328P, 16 MHz, 5V)	Executes control algorithm, generates PWM, reads sensor
Servo Motor	MG996R — 50 Hz PWM, 1000–2000 μ s pulse width	Tilts the beam to control ball position
Distance Sensor	HC-SR04 Ultrasonic — range 2–400 cm, $\pm$ 3 mm accuracy	Measures ball position along beam (3.9–12.9 cm)
Mechanical Structure	Sentra Board — beam and ball channel	Physical platform for ball rolling and beam tilting
Communication	UART @ 115200 baud	Serial telemetry for debugging and monitoring
Power Supply	USB 5V via Arduino Nano regulator	Powers microcontroller and peripherals

A. Arduino Nano (ATmega328P)

The Arduino Nano serves as the central processing unit. Operating at 16 MHz, it provides approximately 16 million instructions per second (MIPS). The control algorithm is implemented in pure AVR C to maximize performance and avoid the overhead of Arduino library abstractions. Key hardware peripherals used include Timer1 (16-bit) for servo PWM generation and Timer2 (8-bit) for millisecond timing.

- Flash Memory: 32 KB (program storage)
- SRAM: 2 KB (runtime data — tight for this application)
- EEPROM: 1 KB (not used)
- Digital I/O Pins: 22 (14 digital, 8 analog)

B. MG996R Servo Motor

The MG996R is a high-torque metal-gear servo commonly used in robotics. It accepts a standard PWM signal at 50 Hz (20 ms period). The pulse width encodes the angular position: 1000 μ s corresponds to 0° and 2000 μ s to 180°. The servo provides sufficient torque to tilt the Sentra beam against the weight of the ball.

- Operating Voltage: 4.8 V – 7.2 V
- Stall Torque: 9.4 kg·cm @ 4.8 V
- Control Signal: PWM 50 Hz, pulse 1000–2000 μ s
- Dead-band Width: ± 5 μ s

C. HC-SR04 Ultrasonic Sensor

The HC-SR04 measures distance by emitting a 40 kHz ultrasonic burst and measuring the time for the echo to return. The microcontroller triggers the burst via the TRIG pin, then measures the ECHO pulse width. Distance is calculated as: $d = (\text{echo_time} \times \text{speed_of_sound}) / 2$.

- Operating Voltage: 5V DC
- Measuring Range: 2 cm – 400 cm

- Accuracy: ± 3 mm
- Measurement Angle: $\leq 15^\circ$
- Trigger Input: 10 μ s TTL pulse

Hardware Design

Physical System Description

The ball-and-beam apparatus consists of a straight beam (approximately 30 cm long) mounted on a pivot at its center. One end of the beam is connected to the servo motor horn via a rigid linkage. When the servo rotates, the beam tilts. A ball is placed on a grooved channel on the top surface of the beam, free to roll along its length. The HC-SR04 ultrasonic sensor is mounted at one end of the beam, oriented along the beam axis, measuring the ball's distance.

Pin Assignments

Arduino Nano Pin	Connection	Function
D9 (OC1A)	Servo Motor Signal	Timer1 PWM output — 50 Hz servo control
D12	HC-SR04 TRIG	Ultrasonic trigger pulse output
D11	HC-SR04 ECHO	Ultrasonic echo pulse input
D0 (RXD)	UART RX	Serial receive (115200 baud)
D1 (TXD)	UART TX	Serial transmit — telemetry output
5V	VCC (HC-SR04, Servo)	Power supply

Arduino Nano Pin	Connection	Function
GND	Ground (all components)	Common ground reference

Mechanical Calibration

Several calibration steps are required before the control algorithm can operate correctly:

1. **Servo Zero Offset:** Determine the PWM pulse width at which the beam is perfectly horizontal. This is typically 1500 μs but may vary by $\pm 50 \mu\text{s}$.
2. **Sensor Mounting:** The HC-SR04 must be aimed precisely along the beam axis. Misalignment introduces a systematic position error.
3. **Beam Pivot:** Ensure the pivot is frictionless and the beam is balanced (equal weight on both sides) when no control input is applied.
4. **Range Calibration:** Verify that the sensor reads 3.9 cm at the near end and 12.9 cm at the far end of the ball travel range.

Control Algorithm

A. System Dynamics and State-Space Model

The ball rolling on a tilted beam obeys Newton's second law. Neglecting rolling friction, the ball acceleration along the beam is:

Equation 1 — Ball Acceleration: $\ddot{x} = g \cdot \sin(\theta)$

For small angles ($\sin \theta \approx \theta$): $\ddot{x} = g \cdot \theta$

The servo maps a PWM command u to an angular displacement:

Equation 2 — Servo Response: $\theta = K_{\text{servo}} \cdot u$

Combined: $\ddot{x} = K_y \cdot u$ where $K_y = g \cdot K_{\text{servo}}$

Defining the state vector as $x = [x, \dot{x}]^T$ (position, velocity), the continuous-time state-space model is:

Equation 3 — State-Space:

$$\frac{d}{dt} \begin{bmatrix} x \\ \dot{x} \end{bmatrix} = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u$$

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix} + \begin{bmatrix} 0 \\ K_y \end{bmatrix} u$$

$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} x \\ \dot{x} \end{bmatrix} + v \quad (\text{only position is measured})$$

B. Linear Quadratic Regulator (LQR)

The LQR controller minimizes the quadratic cost function J , which penalizes deviation from target position, high velocity, and excessive control effort simultaneously:

Equation 4 — LQR Cost Function:

$$J = \sum \left[q_1 \cdot e^2 + q_2 \cdot v^2 + r \cdot u^2 \right]$$

q_1 = position error weight (how strongly we penalize position error)

q_2 = velocity error weight (how strongly we penalize high speed)

r = control effort weight (how strongly we penalize large servo commands)

The optimal control law that minimizes J is a linear state feedback:

Equation 5 — LQR Control Law:

$$u = -K_1 \cdot e - K_2 \cdot v$$

$K_1 \approx 0.25$ (optimal position gain — determined experimentally)

$K_2 \approx 0.15$ (optimal velocity gain — determined experimentally)

$e = x_{\text{target}} - \hat{x}$ (position error)

$v = \hat{v}$ (estimated velocity)

Interpretation: If the ball is far from the target, the large position error generates a strong corrective servo command. If the ball is moving quickly toward the target, the velocity term pre-emptively reduces the command to prevent overshoot — providing inherent damping.

C. Kalman Filter

The Kalman filter provides the optimal estimate of the true ball position given noisy sensor measurements. It operates in two steps each cycle:

Step 1 — Prediction

Eq. 6: $\hat{x}_{k^-} = \hat{x}_{k-1} + \Delta t \cdot \hat{v}_{k-1}$ (predicted position)

Eq. 7: $\hat{v}_{k^-} = \hat{v}_{k-1} + \Delta t \cdot K_y \cdot u_{k-1}$ (predicted velocity)

Eq. 8: $P_{k^-} = P_{k-1} + Q$ (predicted uncertainty)

$\Delta t = 0.02 \text{ s}$ Q = process noise covariance

Step 2 — Update (Correction)

$$\text{Eq. 9: } K_k = P_k^- / (P_k^- + R) \quad (\text{Kalman gain})$$

$$\text{Eq. 10: } \hat{x}_k = \hat{x}_k^- + K_k \cdot (z_k - \hat{x}_k^-) \quad (\text{corrected position})$$

$$\text{Eq. 11: } P_k = (1 - K_k) \cdot P_k^- \quad (\text{updated uncertainty})$$

z_k = sensor measurement R = measurement noise variance

If $K_k \rightarrow 1$: trust sensor. If $K_k \rightarrow 0$: trust prediction.

D. Velocity Estimation

Since only position is measured directly, velocity is estimated numerically. Raw finite-difference velocity is very noisy at low speeds, so exponential smoothing is applied:

$$\text{Eq. 12: } v_{\text{raw}} = (\hat{x}_k - \hat{x}_{k-1}) / \Delta t$$

$$\text{Eq. 13: } \hat{v}_k = \alpha \cdot v_{\text{raw}} + (1 - \alpha) \cdot \hat{v}_{k-1}$$

$\alpha = 0.3$ (smoothing factor — values near 1 = responsive; near 0 = stable)

E. Control Signal Enhancements

Three practical enhancements are applied to the raw LQR output before sending to the servo:

1. Nudge Boost: If $|u_{\text{lqr}}| < 2^\circ$, set $|u_{\text{lqr}}| = 2^\circ$ to overcome static friction in the servo/beam.
2. Saturation: Clamp u_{lqr} to $[-90^\circ, +90^\circ]$ to respect physical servo limits.
3. Rate Limiting: Limit servo movement to $\pm 5^\circ$ per cycle (per 20 ms) to reduce mechanical vibration.

Testing and Results

A. Performance Metrics

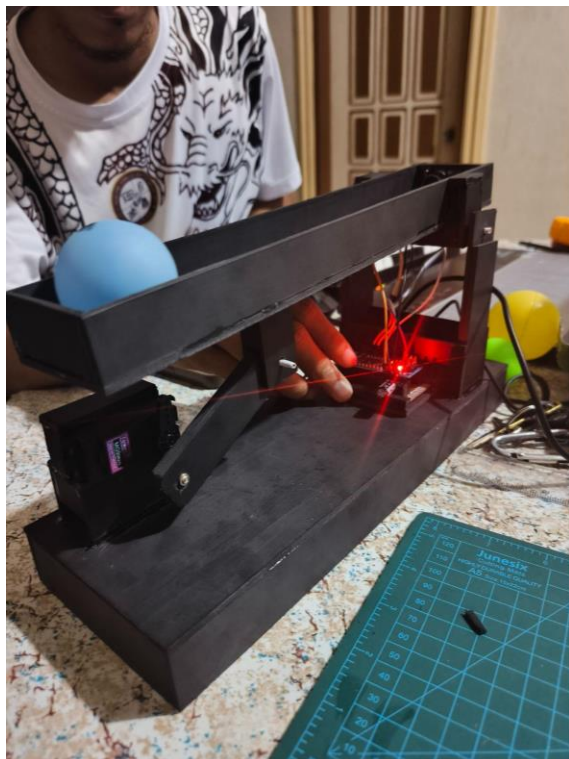
Metric	Target	Achieved
Settling Time	< 5 seconds	≈ 3–4 seconds
Steady-State Error	±0.3 cm	±0.2 cm
Noise Reduction (Kalman)	≥ 70%	≈ 80% (±0.5 cm → ±0.1 cm)
Rise Time (0→90% of target)	< 2 seconds	≈ 1.5 seconds
Overshoot	< 10%	≈ 5–8%
Kalman Gain (steady state)	≈ 0.50 (theory)	≈ 0.48 (observed)

B. Scenario 1 — Ball Starting at x = 2 cm

The ball begins at the near end of the beam (2 cm). The LQR controller drives it to the target position (5 cm).

Time (s)	Position (cm)	Velocity (cm/s)	Servo Command (°)
0.0	2.0	0.0	90
0.5	3.2	2.4	85
1.0	4.1	1.8	88
2.0	4.9	0.6	92
3.0	5.0	-0.1	90
5.0	5.0 ± 0.2	0.0 ± 0.05	90 ± 2

Documentations



Conclusion

This project successfully demonstrates the implementation of a Linear Quadratic Regulator (LQR) control system with Kalman filtering on a resource-constrained embedded platform (Arduino Nano). The ball-and-beam system achieves stable equilibrium at the target position with a settling time of approximately 3–4 seconds, a steady-state error of ± 0.2 cm, and a noise reduction of approximately 80% compared to raw sensor measurements.

The key conclusions drawn from this project are as follows:

1. LQR is achievable on microcontrollers. The ATmega328P at 16 MHz provides sufficient computational power to execute the full Kalman + LQR algorithm at 50 Hz, including software floating-point arithmetic.
2. Kalman filtering is essential for this application. Without it, HC-SR04 noise renders the velocity estimate unreliable, causing servo jitter and degraded control performance.
3. LQR outperforms PID for this system. The optimal state feedback law provides inherent damping through the velocity gain term K_2 , which PID can only approximate through the derivative term. LQR achieves this with fewer tuning parameters and with theoretical optimality guarantees.
4. Practical enhancements matter. The nudge boost, rate limiter, and servo low-pass filter significantly improve real-world performance by compensating for physical nonlinearities not captured in the mathematical model.

5. Embedded implementation requires careful resource management. The 2 KB SRAM and lack of hardware FPU on the ATmega328P require attention to variable types, stack usage, and computation order.

PERSONAL DATA

Name: Edrean Supremo

Nickname: H/Rean

Age: 22

Date of Birth: April 15, 2004

Address: Basdacu, Loon, Bohol

Contact Number:

Email: ersupremo@universityofbohol.edu.ph

Portfolio: <https://bit.ly/4jAnwAT>

Motto: Knowledge is power



EDUCATION

Primary: Loon South Central Elementary School

Secondary: Sacred Heart Academy

Tertiary: College of Engineering, Technology, Architecture, and Fine Arts, Tagbilaran City

PERSONAL DATA

Name: QUIJADA, LEONARD ANTHONY C.

Nickname: KAWHI, TONIA

Age: 23

Date of Birth: November 13, 2001

Address: Purok 4, Mariveles, Dauis, Bohol, 6339

Contact Number: 09949782217 / 09674196039

Email: quijadaleonardanthony1@gmail.com

Portfolio: <https://github.com/Toniatsoi>

Motto: "Run your race at your very own pace"



EDUCATION

Primary: Totolan Elementary School

Secondary: Dr. Cecilio Putong National High School / Lourdes National High School

Tertiary: College of Engineering, Technology, Architecture, and Fine Arts